

Software de Gestão de Pilotos de Automobilismo

Class: 1DD Grupo 3

ISEP — Instituto Superior de Engenharia do Porto

LETI — Licenciatura em Engenharia de Telecomunicações e Informática | 2025/26

Abstract. Este documento apresenta a análise de requisitos e a arquitetura de um sistema de gestão de pilotos de automobilismo, desenvolvido como aplicação de linha de comandos (CLI) em C++. O projeto encontra-se organizado em três iterações dentro da pasta Documentation, sendo a Iteração 01 a que contém os artefactos UML produzidos nesta fase: diagrama de domínio, casos de uso, diagramas de sequência do sistema (SSD), diagrama de classes e diagrama de pacotes. Os dados são persistidos em ficheiros binários, seguindo as restrições da unidade curricular Fundamentos de Desenvolvimento de Software (FSOFT) do ISEP.

Keywords: automobilismo, gestão de pilotos, C++, CLI, UML, análise de requisitos, PlantUML.

1 Contexto / Problema

1.1 Propósito

O presente trabalho consiste no desenvolvimento de uma aplicação de gestão de pilotos de automobilismo em C++, com interface de linha de comandos (CLI). O sistema permite registar pilotos, equipas, veículos e corridas, associar pilotos a corridas através de participações, e consultar as classificações resultantes.

1.2 Funcionalidades Principais

- Criação e listagem de pilotos, equipas, veículos e corridas.
- Registo de participações de pilotos em corridas (posição final e tempo).
- Consulta de classificações por corrida, ordenadas por posição.
- Persistência automática de todos os dados em ficheiros binários.

1.3 User Stories

1. Como utilizador, quero criar um piloto, para que este possa ser registado no sistema.
2. Como utilizador, quero criar uma equipa e associar-lhe pilotos e veículos, para organizar os participantes.
3. Como utilizador, quero registar a participação de um piloto numa corrida, para guardar os resultados.
4. Como utilizador, quero consultar a classificação de uma corrida, para saber a ordenação dos pilotos.

1.4 Requisitos Não Funcionais

Tabela 1. Requisitos Não Funcionais.

Categoria	Descrição
Usabilidade	Interface de linha de comandos.
Suportabilidade	Testes unitários sobre classes de domínio e controladores (pasta ProjectTester).
Restrições de desenho	Dados persistidos em ficheiros binários.
Restrições de implementação	Linguagem C++, sem frameworks externas, compilado com CMake.

2 Estrutura do Repositório

O repositório Git está organizado da seguinte forma. A raiz contém o ficheiro .gitignore que exclui pastas geradas pelo CLion e pelo CMake, mantendo apenas os pastas/ficheiros relevantes para o projeto.

```
fsoft2026_1DD_3/
|
| .gitignore
|
| └── Documentation
|     |
|     | └── Iteration01
|     |     |
|     |     | class_diagram.puml
|     |     | domain_model.puml
|     |     | package_diagram.puml
|     |     | use_cases.puml
|     |     | uc1.puml
|     |     | uc2.puml
|     |     | uc3.puml
|     |     | uc4.puml
|     |     | uc5.puml
|     |     | uc6.puml
|     |     | uc7.puml
|     |     | uc8.puml
|     |     | uc9.puml
|     |     | uc10.puml
|     |     |
|     |     | └── Iteration02
|     |     |     |
|     |     |     | relatorio_gestao_pilotos_fsoft.pdf
|     |     |     |
|     |     |     | └── Iteration03
|     |     |
|     |
|     | └── Project
|     |     |
|     |     | CMakeLists.txt
|     |     | main.cpp
|     |     | main.exe
|     |
|     | └── ProjectTester
```

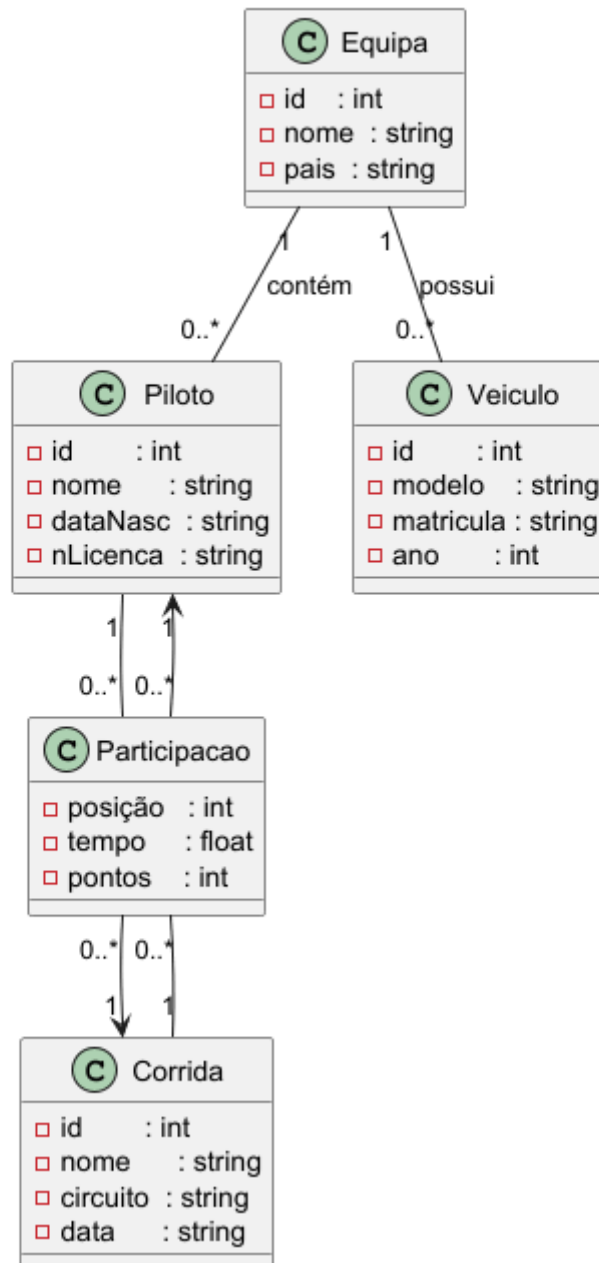
As pastas .idea e cmake-build-debug, são geradas automaticamente pelo CLion e pelo CMake e estão excluídas do repositório através do .gitignore.

3 Modelo de Domínio

O modelo de domínio (ficheiro domain_model.puml) identifica as entidades relevantes do sistema e as relações entre elas, servindo de base para o desenho das classes de implementação.

Figura 1. Modelo de Domínio.

Domain Model - Software de Gestão de Pilotos de Automobilismo



- Uma Equipa contém zero ou mais Pilotos e zero ou mais Veículos.
- Um Piloto pode participar em zero ou mais Corridas, e uma Corrida pode ter zero ou mais Pilotos — esta relação muitos-para-muitos é resolvida pela classe associativa Participação.
- A classe Participação regista a posição final, o tempo e os pontos do piloto nessa corrida.

3.2 Glossário

Tabela 2. *Glossário de termos do domínio.*

Termo	Definição / Descrição
Piloto	Condutor que compete em corridas. Identificado por nome, data de nascimento e número de licença.
Equipa	Organização que agrupa pilotos e veículos. Identificada por nome e país.
Veículo	Automóvel utilizado nas provas. Identificado por modelo, matrícula e ano.
Corrida	Prova de automobilismo com nome, circuito e data.
Participação	Associação entre Piloto e Corrida, incluindo posição final, tempo e pontos obtidos.

4 Requisitos Funcionais

4.1 Diagrama de Casos de Uso (use_cases.puml)

O diagrama de casos de uso apresenta as funcionalidades disponibilizadas ao utilizador dentro do limite do sistema. Os dez casos de uso identificados são:

- UC1 — Criar Piloto
- UC2 — Listar Pilotos
- UC3 — Criar Equipa
- UC4 — Listar Equipas
- UC5 — Criar Veículo
- UC6 — Listar Veículos
- UC7 — Criar Corrida
- UC8 — Listar Corridas
- UC9 — Registar Participação (inclui: Selecionar Piloto, Selecionar Corrida)
- UC10 — Consultar Classificação (inclui: Selecionar Corrida)

4.2 Especificação: UC1 — Criar Piloto

Tabela 3. Especificação de UC1 — Criar Piloto.

Campo	Descrição
Descrição	Adicionar um piloto ao sistema.
Pré-condição	Não existem pré-condições associadas.
Pós-condição	O número de pilotos no sistema aumenta uma unidade.
Caminho básico	1. O Utilizador introduz os dados do piloto (nome, data de nasc., licença). 2. O Utilizador confirma a operação. 3. O Sistema valida os dados. 4. O Sistema armazena o piloto. 5. O Sistema devolve sucesso.
Caminho alternativo	Dados inválidos → O Sistema devolve falha.

4.3 Especificação: UC2 — Listar Pilotos

Tabela 4. Especificação de UC2 — Listar Pilotos.

Campo	Descrição
Descrição	Mostrar todos os pilotos registados no sistema.
Pré-condição	Existem pilotos registados no sistema.
Pós-condição	A lista de pilotos é apresentada ao utilizador.
Caminho básico	1. O Utilizador solicita a listagem de pilotos. 2. O Sistema recebe o pedido. 3. O Sistema obtém todos os pilotos registados. 4. O Sistema apresenta a lista de pilotos ao Utilizador.
Caminho alternativo	Não existem pilotos registados → O Sistema devolve uma lista vazia.

4.4 Especificação: UC3 — Criar Equipa

Tabela 5. Especificação de UC3 — Criar equipa.

Campo	Descrição
Descrição	Adicionar uma nova equipa ao sistema.
Pré-condição	Não existem pré-condições associadas.
Pós-condição	O número de equipas no sistema aumenta uma unidade.
Caminho básico	1. O Utilizador introduz os dados da equipa (nome, país). 2. O Utilizador confirma a operação. 3. O Sistema valida os dados. 4. O Sistema armazena a equipa. 5. O Sistema devolve sucesso.
Caminho alternativo	Dados inválidos → O Sistema devolve falha.

4.5 Especificação: UC4 — Listar Equipas

Tabela 6. Especificação de UC4 — Listar Equipas.

Campo	Descrição
Descrição	Listar todas as equipas registadas no sistema.
Pré-condição	Existem equipas registadas no sistema.
Pós-condição	A lista de equipas é apresentada ao utilizador.
Caminho básico	1. O Utilizador solicita a listagem de equipas. 2. O Sistema recebe o pedido. 3. O Sistema obtém todas as equipas registadas. 4. O Sistema apresenta a lista de equipas ao Utilizador.
Caminho alternativo	Não existem equipas registadas → O Sistema devolve uma lista vazia.

4.6 Especificação: UC5 — Criar Veículo

Tabela 7. Especificação de UC5 — Criar Veículo.

Campo	Descrição
Descrição	Adicionar um novo veículo ao sistema.
Pré-condição	Não existem pré-condições associadas.
Pós-condição	O número de veículos no sistema aumenta uma unidade.
Caminho básico	1. O Utilizador introduz os dados do veículo (modelo, matrícula e ano). 2. O Utilizador confirma a operação. 3. O Sistema valida os dados. 4. O Sistema armazena o veículo. 5. O Sistema devolve sucesso.
Caminho alternativo	Dados inválidos → O Sistema devolve falha.

4.6 Especificação: UC6 — Listar Veículos

Tabela 8. Especificação de UC6 — Listar Veículos.

Campo	Descrição
Descrição	Listar todos os veículos registados no sistema.
Pré-condição	Existem veículos registados no sistema.
Pós-condição	A lista de veículos é apresentada ao utilizador.
Caminho básico	1. O Utilizador solicita a listagem de veículos. 2. O Sistema recebe o pedido. 3. O Sistema obtém todos os veículos registados. 4. O Sistema apresenta a lista de veículos ao Utilizador.
Caminho alternativo	Não existem veículos registados → O Sistema devolve uma lista vazia.

4.7 Especificação: UC7 — Criar Corrida

Tabela 9. Especificação de UC7 — Criar Corrida.

Campo	Descrição
Descrição	Adicionar uma nova corrida ao sistema.
Pré-condição	Não existem pré-condições associadas.
Pós-condição	O número de corridas no sistema aumenta uma unidade.
Caminho básico	1. O Utilizador introduz os dados da corrida (nome, circuito e data). 2. O Utilizador confirma a operação. 3. O Sistema valida os dados. 4. O Sistema armazena a corrida. 5. O Sistema devolve sucesso.
Caminho alternativo	Dados inválidos → O Sistema devolve falha.

4.8 Especificação: UC8 — Listar Corridas

Tabela 10. Especificação de UC8 — Listar Corridas.

Campo	Descrição
Descrição	Listar todas as corridas registadas no sistema.
Pré-condição	Existem corridas registadas no sistema.
Pós-condição	A lista de corridas é apresentada ao utilizador.
Caminho básico	1. O Utilizador solicita a listagem de corridas. 2. O Sistema recebe o pedido. 3. O Sistema obtém todas as corridas registadas. 4. O Sistema apresenta a lista de corridas ao Utilizador.
Caminho alternativo	Não existem corridas registadas → O Sistema devolve uma lista vazia.

4.9 Especificação: UC9 — Registar Participação

Tabela 11. Especificação de UC9 — Registar Participação.

Campo	Descrição
Descrição	Associar um piloto a uma corrida, registando posição e tempo.
Pré-condição	O piloto e a corrida devem existir no sistema.
Pós-condição	Uma nova participação é adicionada ao sistema.
Caminho básico	1. O Utilizador pede a lista de pilotos. 2. O Utilizador seleciona um piloto. 3. O Utilizador pede a lista de corridas. 4. O Utilizador seleciona uma corrida. 5. O Utilizador introduz posição e tempo. 6. O Sistema valida os dados. 7. O Sistema guarda a participação. 8. O Sistema devolve sucesso.
Caminho alternativo	Dados inválidos → O Sistema devolve falha.

4.10 Especificação: UC10 — Consultar Classificação

Tabela 12. Especificação de UC10 — Consultar Classificação.

Campo	Descrição
Descrição	Consultar a classificação de uma corrida registada no sistema.
Pré-condição	Existem corridas registadas no sistema.
Pós-condição	A classificação da corrida selecionada é apresentada ao utilizador.
Caminho básico	1. O Utilizador solicita a lista de corridas. 2. O Sistema apresenta a lista de corridas disponíveis. 3. O Utilizador seleciona uma corrida através do seu identificador. 4. O Sistema obtém as participações associadas à corrida selecionada. 5. O Sistema ordena as participações pela posição. 6. O Sistema apresenta a classificação ao Utilizador.
Caminho alternativo	Não existem participações registadas para a corrida selecionada → O Sistema devolve “sem resultados”.

5 Requisitos não Funcionais

Os **requisitos não funcionais** descrevem *como o sistema deve funcionar* — desempenho, segurança, usabilidade, etc.

5.1 Usabilidade

- O sistema deve apresentar uma interface simples e intuitiva para o utilizador.
- As mensagens de erro devem ser claras e compreensíveis.

5.2 Desempenho

- O tempo de resposta para listar pilotos/equipas/corridas não deve ser longo.

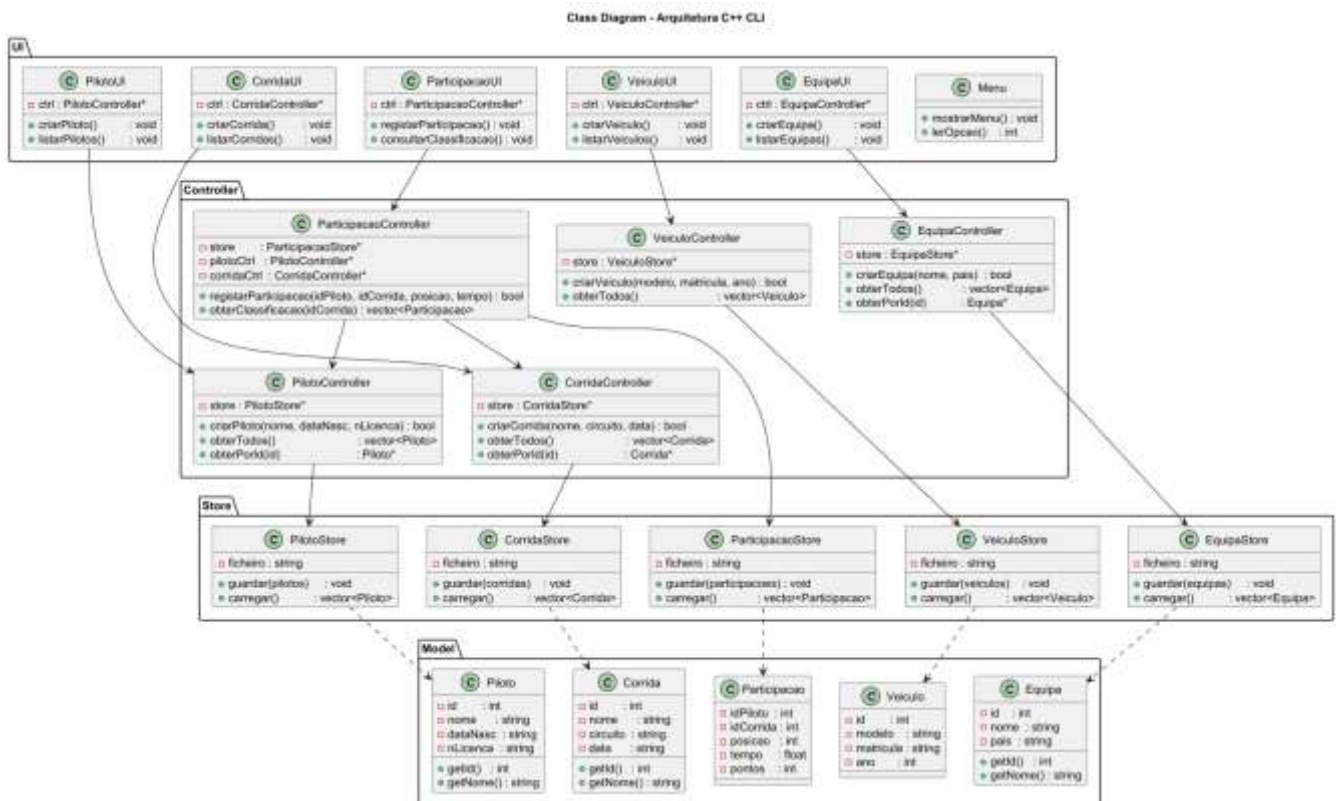
- O sistema deve suportar pelo menos **100 utilizadores simultâneos** sem perda significativa de desempenho.

5.3 Integridade dos Dados

- Não podem existir pilotos duplicados com a mesma licença.
- Não podem existir equipas duplicadas com o mesmo nome.
- Todos os dados inseridos devem ser validados antes de serem guardados.

6 Diagrama de Classes

Figura 2. Diagrama de classes.



7 Arquitetura do Projeto C++

7.1 Organização em Camadas (class_diagram.puml)

O projeto adota uma arquitetura em quatro camadas separadas, refletida no diagrama de classes (class_diagram.puml):

- Model — classes de dados puras: Piloto, Equipa, Veículo, Corrida, Participação.
- Store — classes de persistência em ficheiros binários (*Store.h / *Store.cpp).
- Controller — classes com a lógica de negócio e validação (*Controller.h / *Controller.cpp).
- UI — classes de interação com o utilizador via consola (*UI.h / *UI.cpp) e a classe Menu.

As dependências seguem sempre o sentido UI → Controller → Store → Model, garantindo a separação de responsabilidades e facilitando os testes unitários a implementar na pasta ProjectTester.

7.2 Estrutura de Pacotes (package_diagram.puml)

O diagrama de pacotes (package_diagram.puml) complementa o diagrama de classes ao mostrar a organização física dos ficheiros fonte dentro da pasta Project, as dependências entre pacotes e os ficheiros de dados binários gerados em runtime na pasta data/.

Tabela 13. Pastas do código-fonte (Project/).

Pasta	Conteúdo
model/	Piloto, Equipa, Veiculo, Corrida, Participacao (.h / .cpp)
controller/	PilotoController, EquipaController, VeiculoController, CorridaController, ParticipacaoController (.h / .cpp)
store/	PilotoStore, EquipaStore, VeiculoStore, CorridaStore, ParticipacaoStore (.h / .cpp)
ui/	Menu, PilotoUI, EquipaUI, VeiculoUI, CorridaUI, ParticipacaoUI (.h / .cpp)
data/	Ficheiros binários gerados em runtime (pilotos.bin, equipas.bin, etc.)

Figura 3. Diagrama de Pacotes da Arquitetura do Sistema



Referências

1. Paulo Baltarejo Sousa, Carlos Freitas: Requirements Engineering/Analysis — FSOF TP Week
4. ISEP, Porto (2025).